

Arduino Workshop: *Arduino IDE*

Project Overview

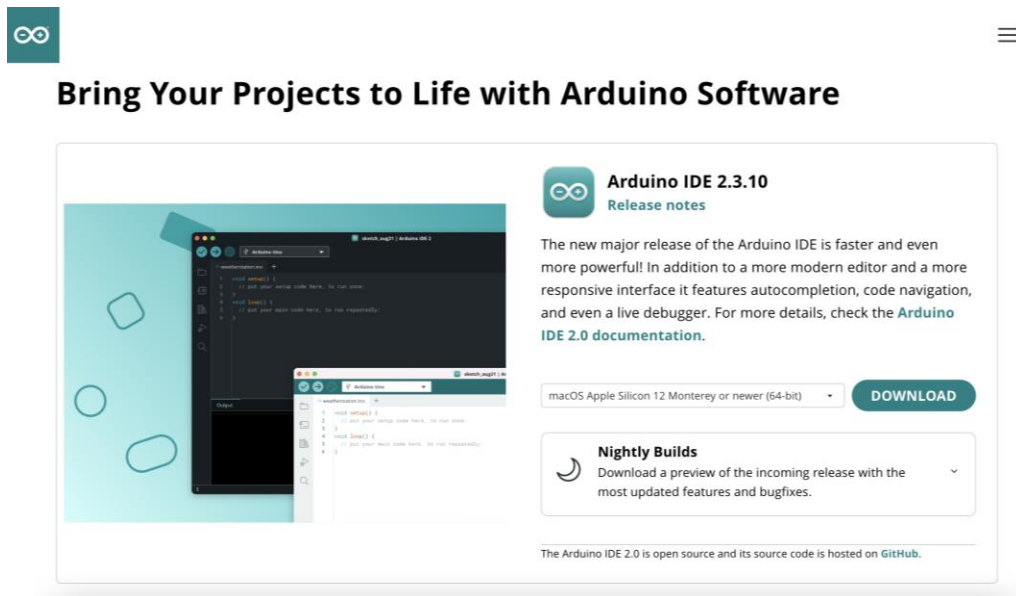
Welcome to the Purdue MIND Arduino Onboarding Project! In this project, you will use an Arduino to interface with LEDs and sensors while learning the fundamentals of embedded programming, circuit integration, and data acquisition.

Throughout the project, you will progress from controlling a simple LED to working with increasingly advanced inputs and sensors, exploring how an Arduino can collect and process different types of real-world data.

Estimated time: 2-3 hours

Arduino IDE Installation

- 1) Navigate to <https://www.arduino.cc/en/software/>
 - a. Select your operating system from the drop-down menu and click “Download”



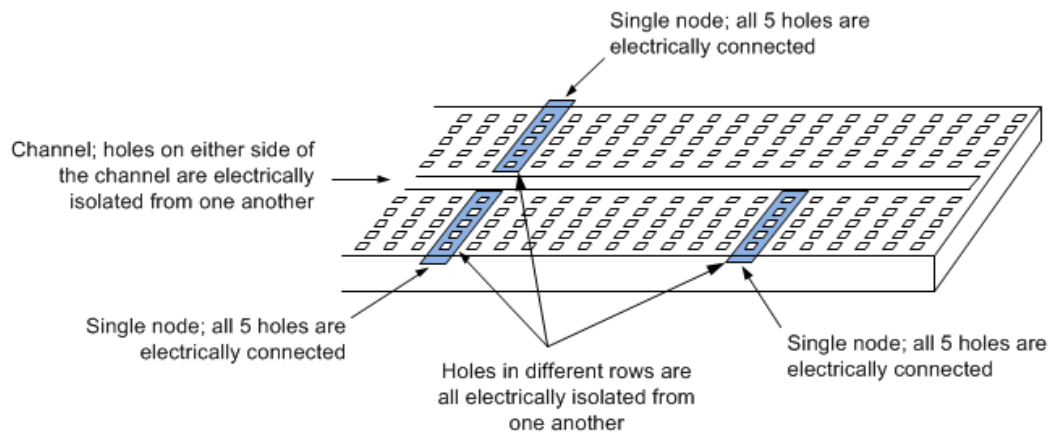
Key Ideas

Please read this section before beginning to build your circuit!

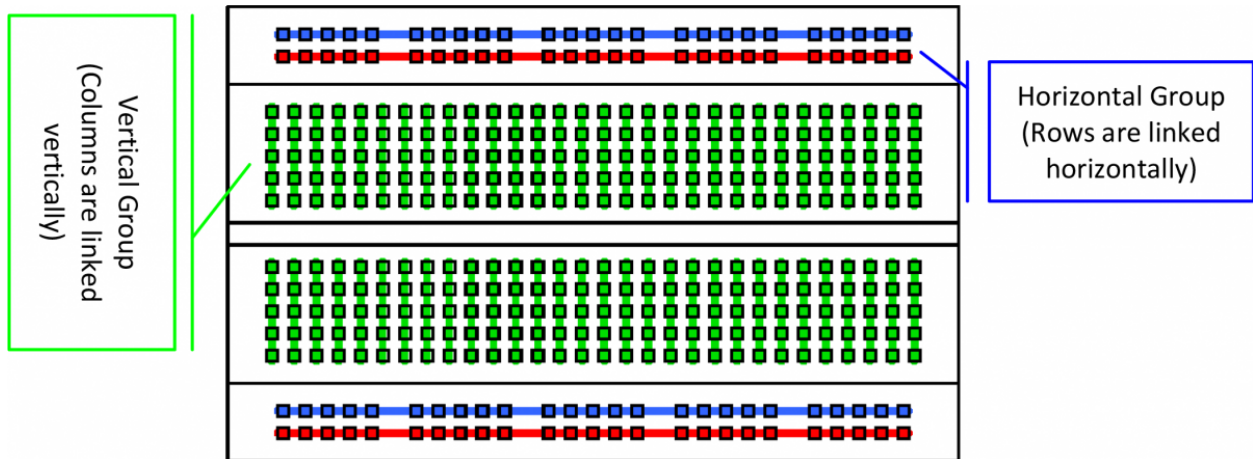
Rather than simply trying to recreate the pictures we provide, the goal is to understand how the connections work and why they are made.

Understanding the basics of your Arduino board will help you troubleshoot problems you will inevitably encounter.

1) Breadboard Holes



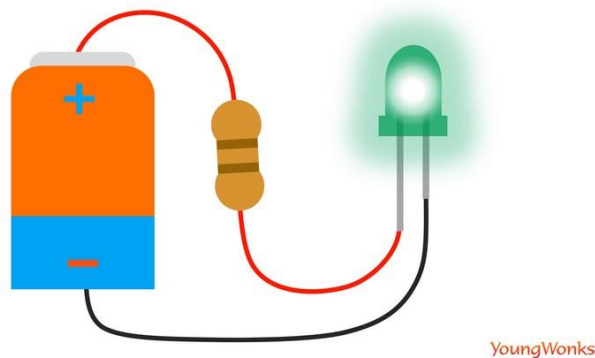
- Holes within the same connected row of a breadboard are electrically connected
 - The **five holes in each numbered row** are connected to one another.
 - BUT, the two sides of the **center channel are NOT connected** to each other.
- Holes within the same connected column of the main section breadboard are NOT electrically connected.
- BUT, the holes along each **power rail columns (+ and -)** ARE connected along the rail. (See image below)



This means your wire **does not need to be placed in the exact same hole shown in the picture**. It just needs to be in a hole that is electrically connected to the correct component.

- Ex: If a wire is shown in hole **A10**, placing it in **B10, C10, D10, or E10** (any open hole in the same row) will make the same connection because all five holes in that row are connected.

2) Power and Ground



- **A power source and GND (ground)** work together to provide a complete path for electricity to flow through your circuit.
 - A **power source supplies electrical energy** to a component. Depending on the circuit, this may come from the Arduino's **5V pin, 3.3V pin, or a digital output pin**.
 - **GND provides a path back to the Arduino**, completing the circuit.

- For a component to work, it generally needs to be connected to **both a power source and ground**.
- If the connection to a power source is missing, the electrical path is incomplete and the component may not work.

Imagine every circuit you build as a loop. Electricity travels from the Arduino's power source pin --> through the component --> back to GND. Every loop needs to be complete for current to flow.

Troubleshooting

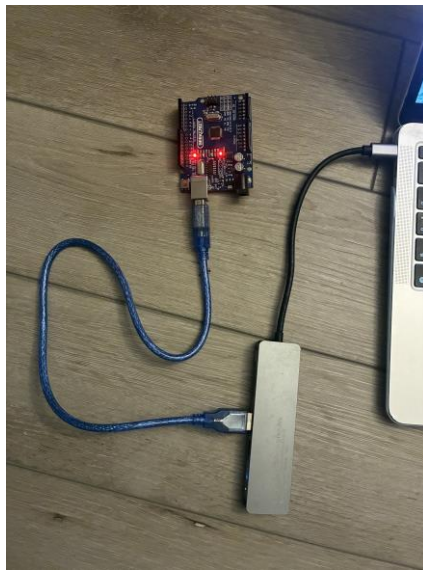
Refer to this section if your circuit is not working as expected.

- **Check your wires**
 - Make sure each jumper wire is pushed all the way into the correct pin and is not loose.
- **Double-check your connections**
 - Compare your circuit to the provided diagram or connection table.
 - Focus on which rows and pins are connected and how the current may flow through components, not whether your circuit looks exactly like the picture.
- **Check your USB connection**
 - Make sure the Arduino is securely connected to your computer and that the board has power.
- **Check the code**
 - If you are getting code errors when you verify any copy-pasted code, check indents, structure, and whether it matches the piece of code we gave you.

- **Still not working?**
 - There are some faulty Arduino boards. Feel free to get a new Arduino board from the front if it seems your original board is faulty, and let MIND leadership know
 - Ask a member of MIND leadership to take a second look at your circuit. We are here to help :)

Board Connection

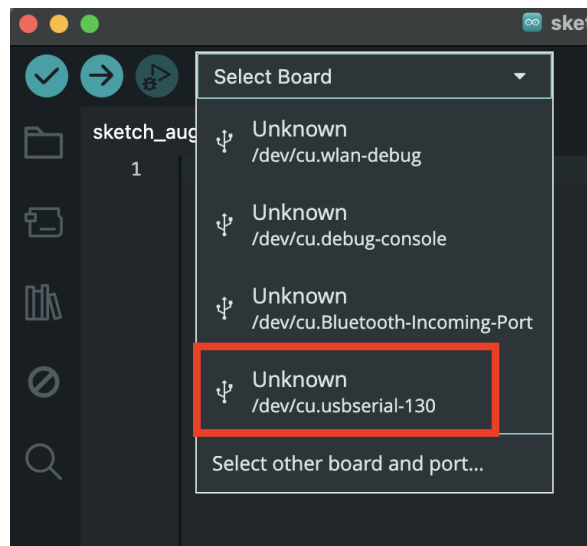
- 1) Collect your Arduino board
 - a. Connect your board to your computer via USB (Grab adapter from the front if needed)



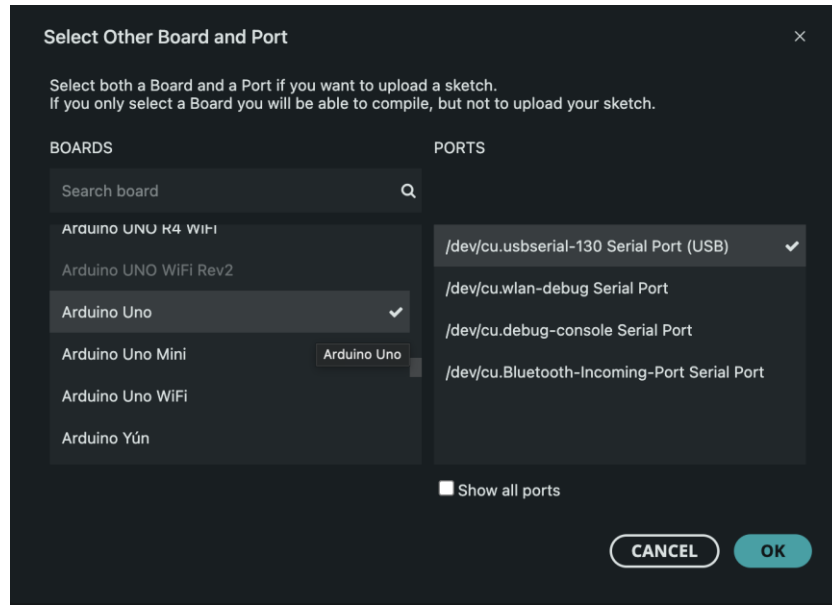
- 2) Open Arduino IDE on your computer
- 3) Navigate to the “Select Board” dropdown menu at the top of your screen and select the correct board.

- a. A connected Arduino typically appears as a **COM, usbserial, or usbmodem** depending on your computer type. Select the matching port to communicate with the board as shown in the image below.
- b. This COM port allows the IDE to communicate with the board
- c. There may be multiple selections depending on how many USB connections you currently have on your laptop. Those other ports are communication connections created by your computer not additional Arduino boards.

Example for Mac:



- 4) Select “**Arduino Uno**” as your board. Then press “OK”.



5) Copy and paste the code below.

- a. The indents may not copy and paste properly. If they do not, manually enter the indents, copying the structure of the text below.

```
const int ledPin = 13; //can be changed to the pin that you attached your LED to

void setup() {
    // put your setup code here, to run once:
    pinMode(LED_BUILTIN, OUTPUT); //LED_BUILTIN refers to the built in LED on the Arduino
    Uno Board which is pin 13 //this can be changed to the ledPin variable initialized earlier
}

void loop() {
    // put your main code here, to run repeatedly:
    digitalWrite(LED_BUILTIN, HIGH);
    delay(1000);
    digitalWrite(LED_BUILTIN, LOW);
    delay(1000);
}
```

Code Explanation:

- 1) `const int ledPin = 13;`
 - Creates a variable called ledPin and assigns it to **digital pin 13**.

- `const` means the value will not change while the program is running.
- `int` means the variable stores an integer (whole number).
- `ledPin` is the name we give the variable.
- 13 is the Arduino pin connected to the LED.

2) `void setup() {`

- The `setup()` function runs once the Arduino is powered on or reset. It is typically used to configure the Arduino before the main program begins.
- Command code is placed inside this space

3) `pinMode(LED_BUILTIN, OUTPUT);`

- This configures the built-in LED pin as an **output**, allowing the Arduino to send a signal to control the LED.

4) `void loop() {`

- Runs **continuously** while the Arduino is powered.
- Command code is placed inside this space
 - Commands execute from top to bottom

5) `digitalWrite(LED_BUILTIN, HIGH);`

- Sets the LED pin **HIGH**, turning the LED on.

6) `delay(1000);`

- Pauses the program for **1000 milliseconds (1 second)**.

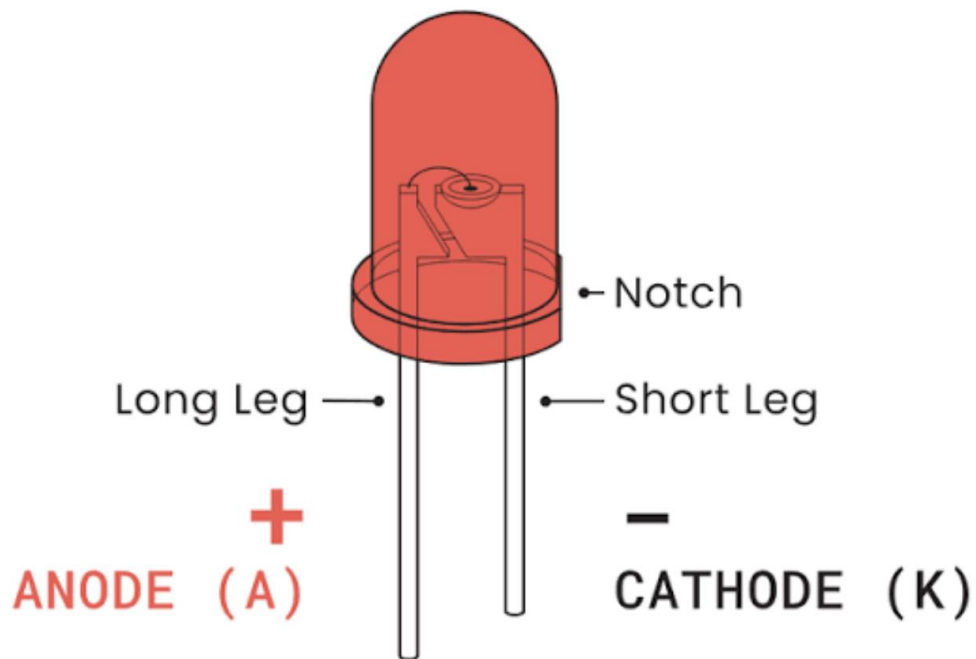
7) `digitalWrite(LED_BUILTIN, LOW);`

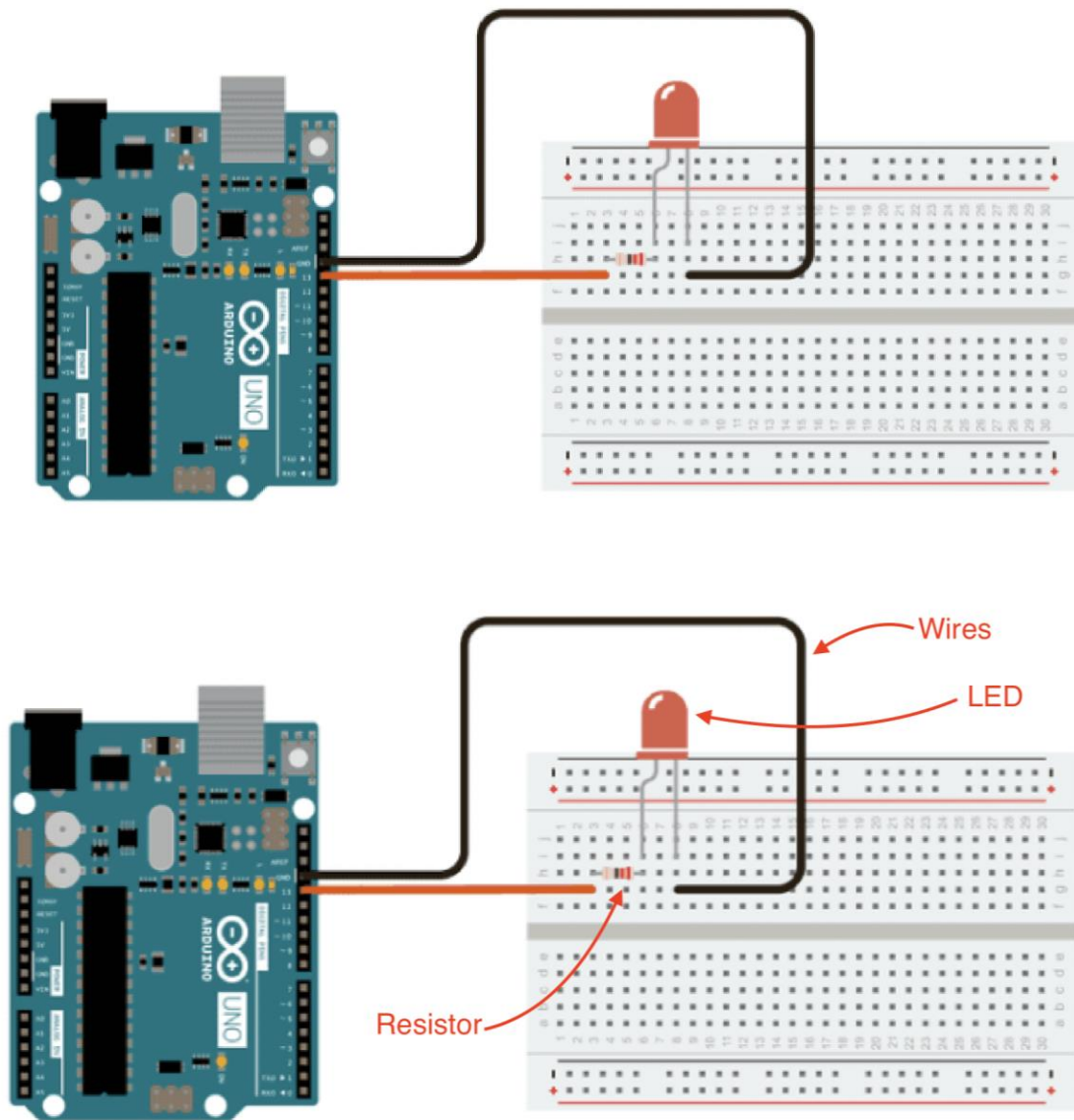
- Sets the LED pin **LOW**, turning the LED off.

8) `delay(1000);`

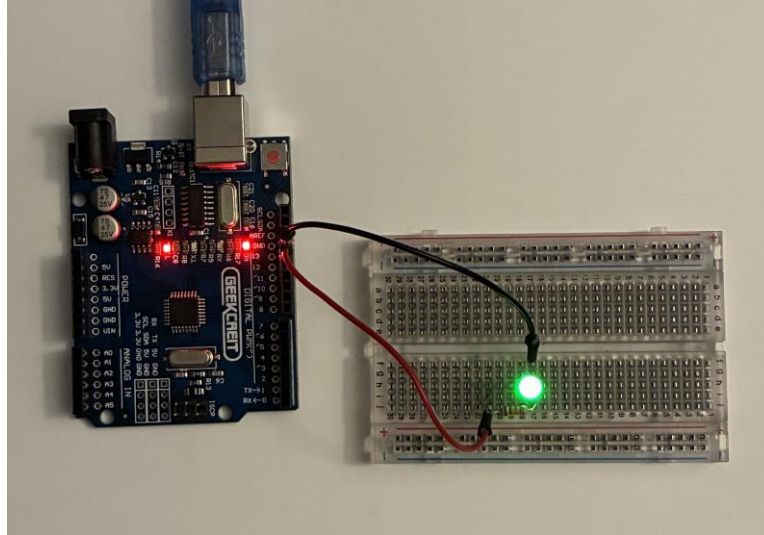
- Waits another 1 second.

- 6) Set up the circuit, as shown in the image below. The resistor is 220 ohms (this value does not have to be exact; just make sure it is around 220!) and the pin is 13.
- Make sure the wire connecting your circuit to pin 13 connects to the POSITIVE side of the LED (anode).
 - LEDs are directional, meaning current can only flow through them in one direction. This means the anode must face the voltage source (pin 13) and the cathode must face ground for current to flow and the LED to light.

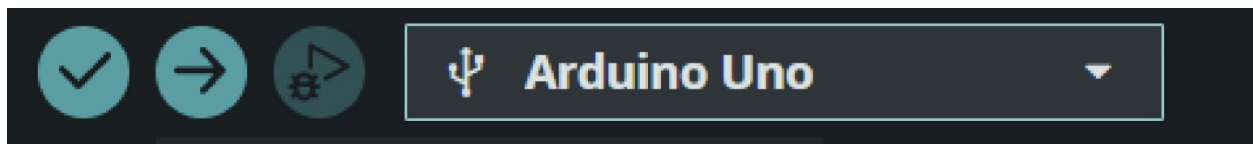




Physical circuit example:



- 7) Press the button shown below: **“Verify”** (checkmark)
 - a. If you receive any **“stray” compilation errors**, this may be the result of the indent issue. Delete and manually re-enter the indentation on the affected lines. (You may need to do this for the commented explanation lines too)
 - b. The spacing errors will show up as clear yellow boxes at the start of the affected lines.
- 8) Press the button shown below: **“Compile”** (arrow)
 - a. **Your circuit should be working!** You should see your LED blinking.



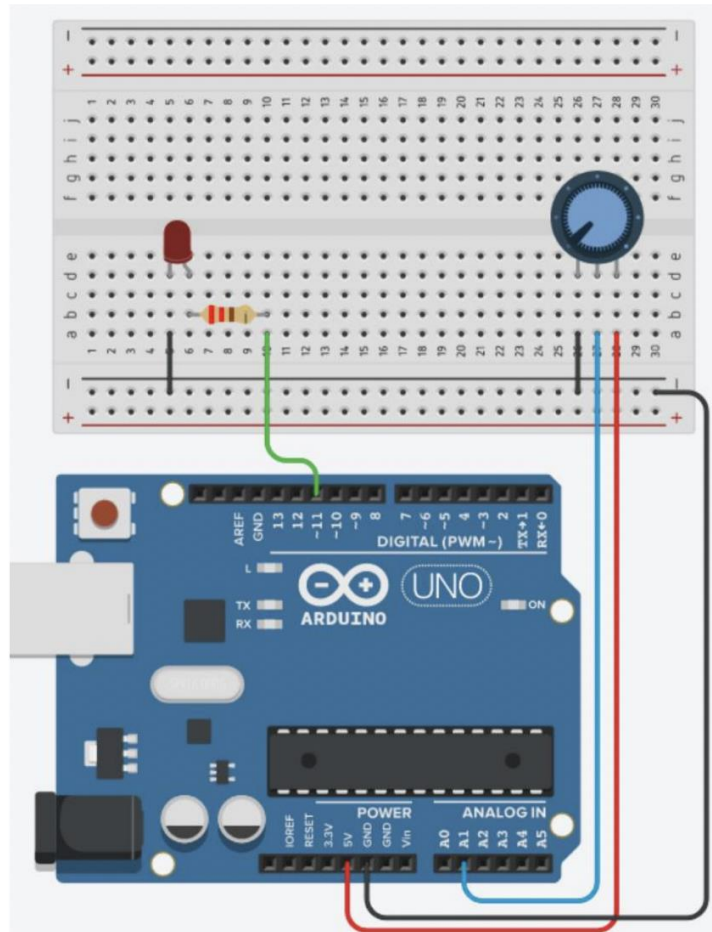
Remember to reference the **“Troubleshooting”** section (page 4) if your circuit is not working/having issues.

Project 1: *Potentiometer (Analog Input)*

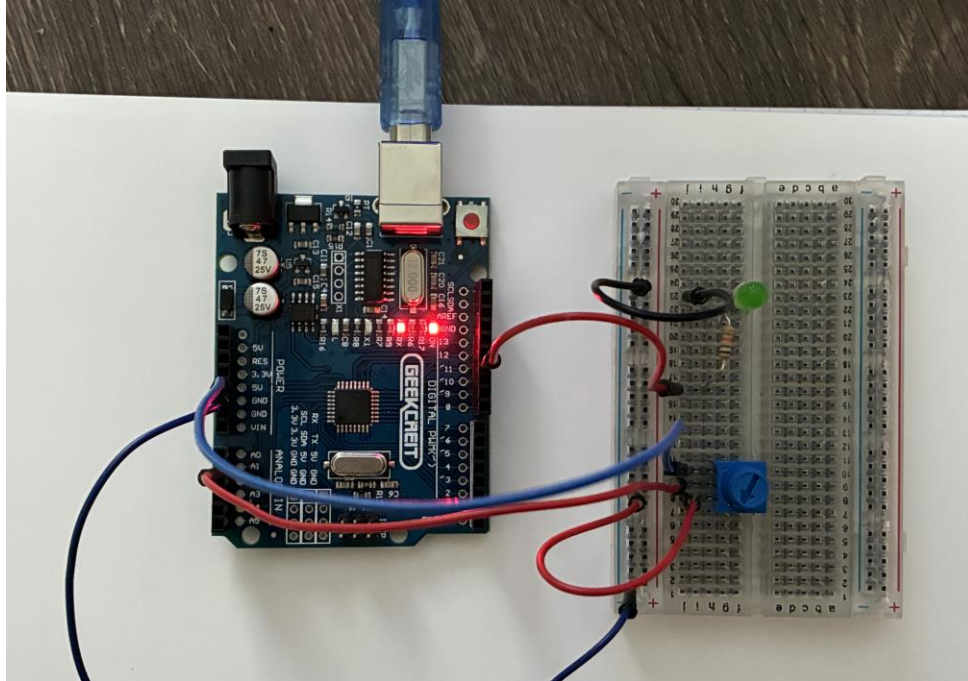
A **potentiometer** is a component with a dial that can be turned to change an electric signal. In other words, it is a variable resistor that can be used to create an adjustable voltage. As you turn the dial, the voltage sent to the Arduino changes.

In this project, the Arduino will measure this changing voltage and use it to control the **brightness of an LED**.

- 1) Set up the circuit using the schematic below (keep the 220 ohms resistor, but switch the LED pin to pin 11 and potentiometer pin at A1) .



Physical circuit example:



2) Copy and paste the code below.

- a. The indents may not copy and paste properly. If they do not, manually enter the indents, copying the structure of the text below.

```
const int ledPin = 11;
const int potentiometer = A1;

void setup() {
    // put your setup code here, to run once:
    pinMode(ledPin, OUTPUT);
}

void loop() {
    // put your main code here, to run repeatedly:
    int potentiometerValue = analogRead(potentiometer); //this value will be from 0-1023, a 10-bit number
    int brightness = map(potentiometerValue, 0, 1023, 0, 255); //to convert the 10-bit number to a byte number
    analogWrite(ledPin, brightness); //this takes byte values or a number in range of 0-255
}
```

Code Explanation:

- Define the components

- `const int ledPin = 11;`

- Creates a variable called ledPin and assigns it to **pin 11**, where the LED is connected.

- `const int potentiometer = A1;`

- Creates a variable called potentiometer and assigns it to **analog pin A1**, where the potentiometer is connected.

- Configure the Arduino

- `void setup() {`

- Begins the setup() function. Everything inside this section runs **once** when the Arduino starts.

- `pinMode(ledPin, OUTPUT);`

-

- Configures the LED pin as an **output**, allowing the Arduino to send a signal to the LED.
 - } : Ends the setup() function.

- Read the Potentiometer

- `void loop() {`

- Begins the loop() function. Everything inside this section will **repeat continuously** while the Arduino is running.

- `int potentiometerValue = analogRead(potentiometer);`

- Reads the analog signal from the potentiometer and stores the reading as potentiometerValue.
 - analogRead() measures the changing signal at an analog input pin and converts it into a numerical value the Arduino can use.

- Convert the Reading

- `int brightness = map(potentiometerValue, 0, 1023, 0, 255);`

- Takes the potentiometer reading from **0–1023** and converts it to a brightness value from **0–255**.

- map() converts the input range into the range needed to control the LED since the potentiometer reading and LED output use **different numerical ranges**.
- Control the LED
 - `analogWrite(ledPin, brightness);`
 - Sends the calculated brightness value to the LED
 - 0 = LED off
 - 255 = maximum output

Essentially, the code takes an analog input from the potentiometer dial as a 10-bit number (range of 0-1023). Then, it maps this number to the range of 0-255 so that we can use the potentiometer value in analogWrite(). This analogWrite() function allows us to control the brightness of the LED attached to this circuit.

- 3) To display the analogRead() value of the potentiometer on Arduino's Serial Monitor, add this line of code to the setup section.

```
Serial.begin(9600);
```

- 4) Add these lines to the loop portion, so that the printed values get updated as the potentiometer is turned.
 - a. NOTE: The difference between print and println is that println will tell the monitor to start the next print on a new line. This is to make the monitor easier to read.

```
Serial.print("Potentiometer Value:");  
Serial.println(potentiometerValue);  
delay(100);
```

- 5) Press **“Verify”**.
 - a. If you receive any **“stray” compilation errors**, this may be the result of the indent issue. Delete and manually re-enter the indentation on the affected lines.

6) Press “**Compile**”.

7) To open the “**Serial Monitor**”, go to “**Tools**” and select **Serial Monitor** or select the **Serial Monitor** icon at the top right.

a. **Serial Monitor:** Displays the sensor readings as **numbers/text**. For this project, this lets you see the potentiometer reading as it changes.

b. Make sure the Serial Monitor baud rate matches the number (9600) set in the Serial.begin() function in the bottom right of the Serial Monitor window.

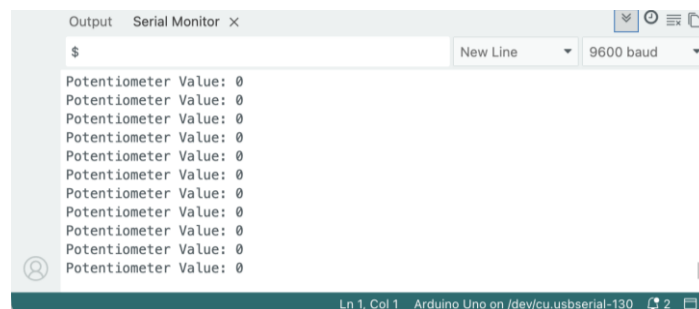
c. Twist the knob of your potentiometer. **You should be able to see your Potentiometer values changing in real time along with the brightness of your LED!**

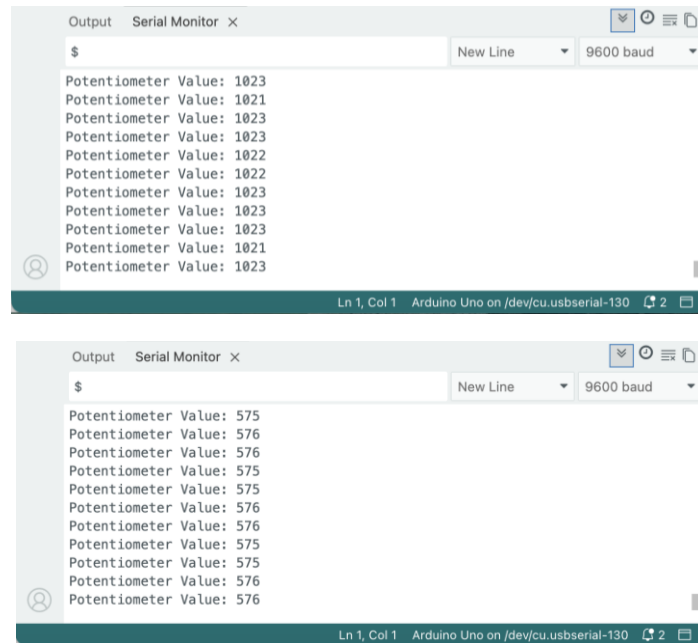
d. **Common issue:** If the values stay constant despite you turning the knob, **your potentiometer may be flipped**. Flip it so the opposite wires are connected to each side and see if there is any change.

8) Open the “**Serial Plotter**”, located in the same place as the Serial Monitor. Twist the knob of your potentiometer. **You should be able to see a live graph of the potentiometer changing!**

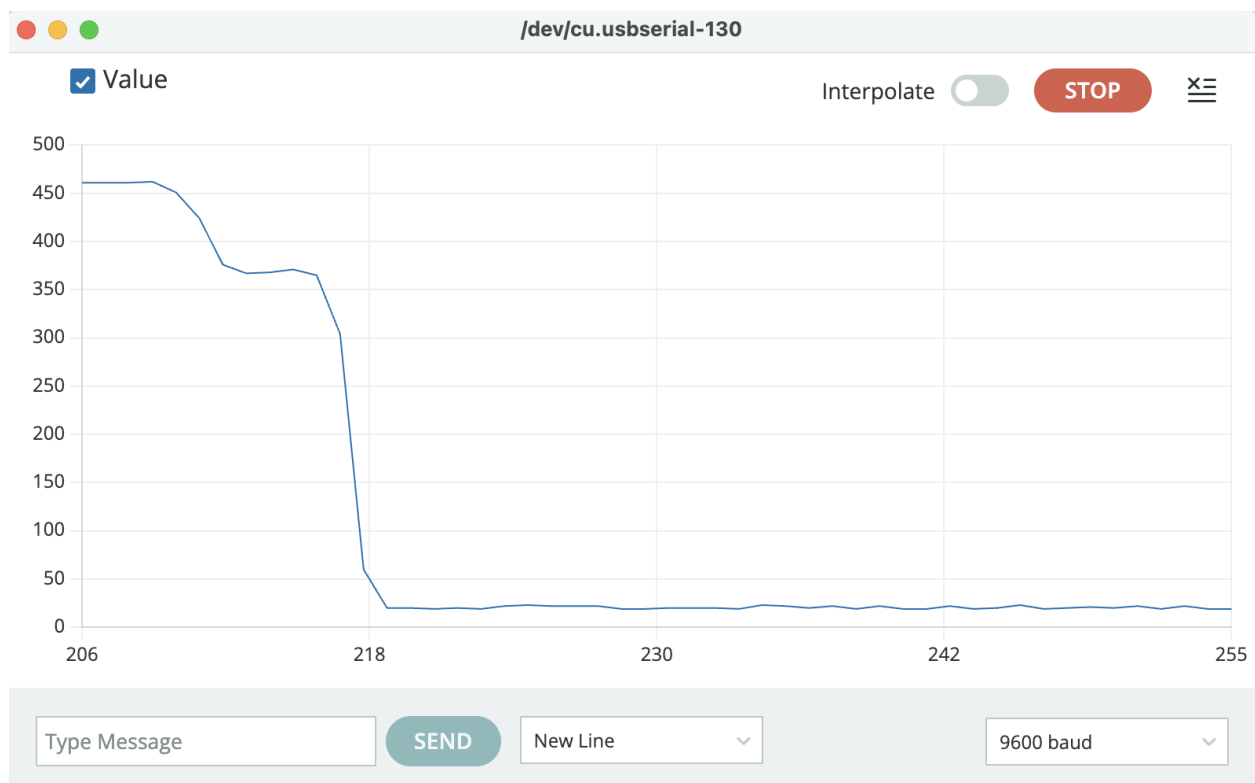
a. **Serial Plotter:** Displays those numerical readings as a **live graph over time**, making it easier to visualize how the potentiometer reading changes over time.

Changing Monitor Values Ex:





Plotter ex:



Remember to reference the “Troubleshooting” section (page 4) if your circuit is not working/having issues.

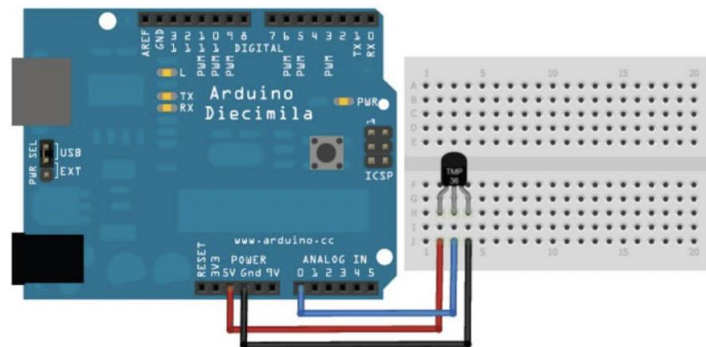
***DO NOT TAKE APART YOUR CIRCUIT! Please find a member of MIND leadership to get checked off for completing this project.**

Project 2: *Temperature Sensor (TMP36)*

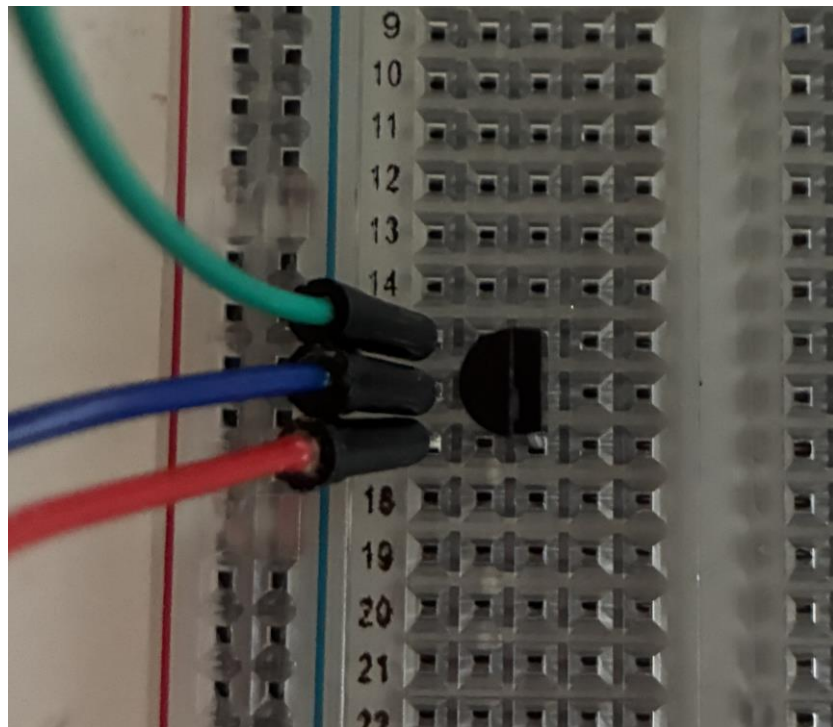
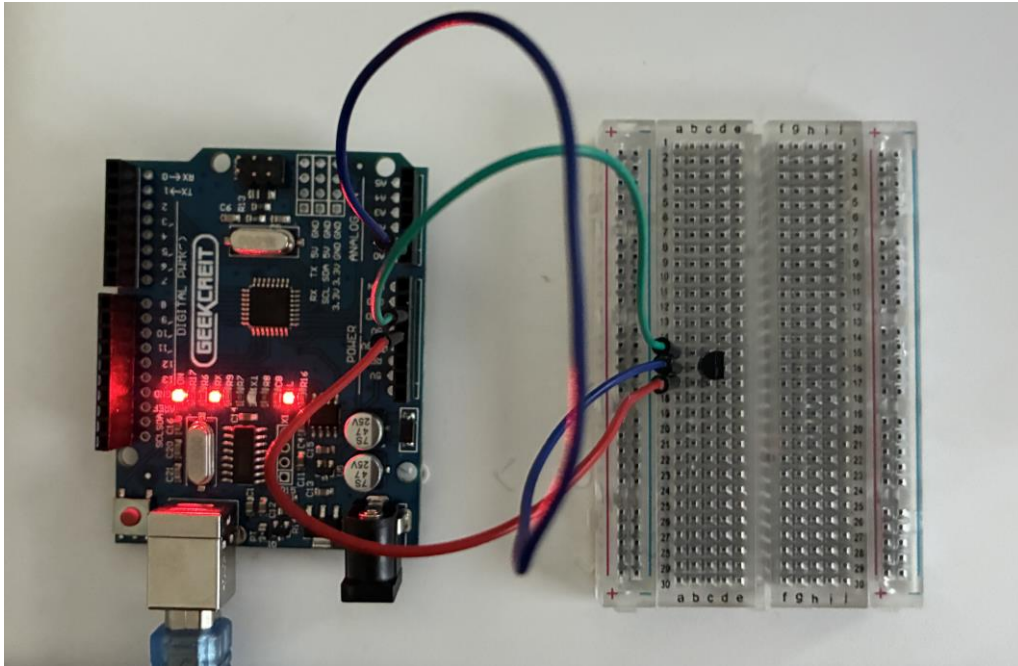
A **temperature sensor** is a component that detects changes in temperature and represents them as an electrical signal. The TMP36 sensor used in this project produces a **changing output voltage based on the temperature it measures**.

In this project, the Arduino will read this changing voltage, **convert it into a temperature in degrees Celsius**, and display the measured temperature using the **Serial Monitor**.

- 1) Use the following schematic to connect the board and temperature sensor together.
 - a. **IMPORTANT:** Ensure that your wires are connected to the rounded side of the temperature sensor! Reference the physical circuit example below.



Physical circuit ex:



2) Copy and paste the code below.

- a. The indents may not copy and paste properly. If they do not, manually enter the indents, copying the structure of the text below.

```
const int tempPin = A0;
void setup() {
    // put your setup code here, to run once:
    Serial.begin(9600);
}

void loop() {
    // put your main code here, to run repeatedly:
    int sensorValue = analogRead(tempPin); //output in range of 0-1023
    float voltage = sensorValue * (5.0 / 1023.0); //converting to volts, it is 5.0
    // because we are using the 5V pin on the Arduino board, if using 3.3V then change
    float celsius = (voltage - 0.5) * 100.0; //converts from 10 mV per degree
    // with a 500 mV offset
    Serial.print("Temperature in Celsius = ");
    Serial.println(celsius);
    delay(1000);
}
```

Code Explanation:

- Define the Temperature Sensor
 - `const int tempPin = A0;`
 - Creates a constant called tempPin and assigns it to **analog pin A0**, where the temperature sensor's output is connected.
- Set up Serial Communication
 - `void setup() {`
 - Begins the setup() function, which runs **once** when the Arduino is powered on or reset.
 - `Serial.begin(9600);`
 - Starts communication between the **Arduino and your computer** at a rate of 9600 bits per second. This allows the Arduino to send the measured temperature to the Serial Monitor.
 - `}`

- 9600 is the **baud rate**, or the speed at which data is communicated. The Serial Monitor must be set to the same baud rate so the computer can correctly interpret the data.
- Read the Temperature Sensor
 - `void loop() {`
 - Begins the loop() function. Everything inside will **repeat continuously** while the Arduino is running.
 - `int sensorValue = analogRead(tempPin);`
 - Reads the analog signal from the temperature sensor at A0 and stores it as sensorValue.
 - At this point, the Arduino does **not know the temperature yet**. It has only converted the voltage coming from the sensor into a numerical value from 0–1023 (same as the last project).
- Convert the Reading to Voltage
 - `float voltage = sensorValue * (5.0 / 1023.0);`
 - Converts the Arduino's **0–1023 reading back into a voltage**.
 - Since the Arduino is using a 5 V reference:
 - $0 = 0\text{ V}$
 - $1023 = 5\text{ V}$
 - float is used here since voltage can contain a decimal. A float allows the program to store numbers with decimal places. int does not.
- Convert Voltage to Temperature
 - `float celsius = (voltage - 0.5) * 100.0;`
 - Converts the measured voltage into a temperature in **degrees Celsius** using the TMP36 sensor's voltage-to-temperature relationship.
 - The TMP36 has a **0.5 V (500 mV) offset** and its output changes by **0.01 V (10 mV) for every 1°C**. This equation converts that voltage into an actual temperature.
- Display the Temperature
 - `Serial.print("Temperature in Celsius = ");`
 - Sends the text **"Temperature in Celsius = "** to the Serial Monitor.

- `Serial.println(celsius);`
 - Prints the calculated temperature after the text.
 - The difference between `print()` and `println()` is that `println()` moves to a **new line afterward**, so the next temperature reading appears on a separate line.
- Wait before taking another reading
 - `Serial.println(celsius);`
 - Pauses the program for **1000 milliseconds (1 second)** before the loop repeats.
 - This means the Arduino displays a new temperature reading approximately **once every second**.

Essentially, for the Arduino to convert the 10-bit analog reading to a temperature, first the value has to be converted to volts. This voltage value is then placed into this formula: Centigrade temperature = [(analog voltage in V) – 0.5] * 100.

- 3) Open the **Serial Monitor** and **Serial Plotter** to see the output of the sensor. Try blowing the sensor to cool it down or pinching it/blowing hot air to heat it up. Watch how the numerical value in the Serial Monitor and the graph in the Serial Plotter change in response.

Serial Monitor ex:



Serial Plotter ex:



Remember to reference the “Troubleshooting” section (page 4) if your circuit is not working/having issues.

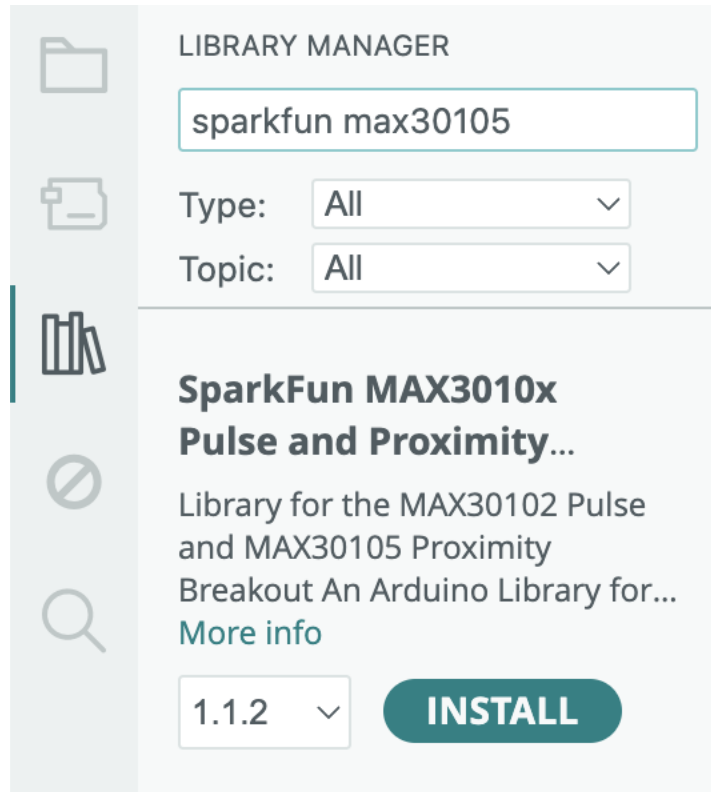
***DO NOT TAKE APART YOUR CIRCUIT! Please find a member of MIND leadership to get checked off for completing this project.**

Project 3: *Pulse Sensor*

A **pulse sensor** is a component that detects changes in blood flow associated with each heartbeat. The MAX30105 sensor used in this project uses **light to detect these changes** and sends the measured data to the Arduino.

In this project, the Arduino will process the sensor data to **detect heartbeats and calculate heart rate in beats per minute (BPM)**. You will also learn how to work with a more advanced sensor and view the resulting measurements through the **Serial Monitor**.

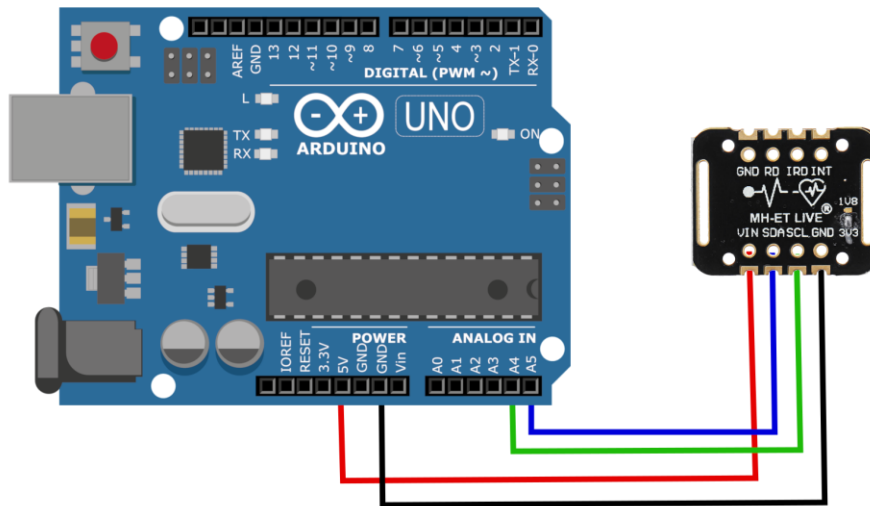
- 1) Open the **Library Manager** using the tab on the left-hand side panel or through the **Tools** dropdown.
- 2) Search for **MAX30105** and install the **1.1.2** version of this library.



- 3) Connect the MAX30102 to the Arduino Uno with wires by using the following table.

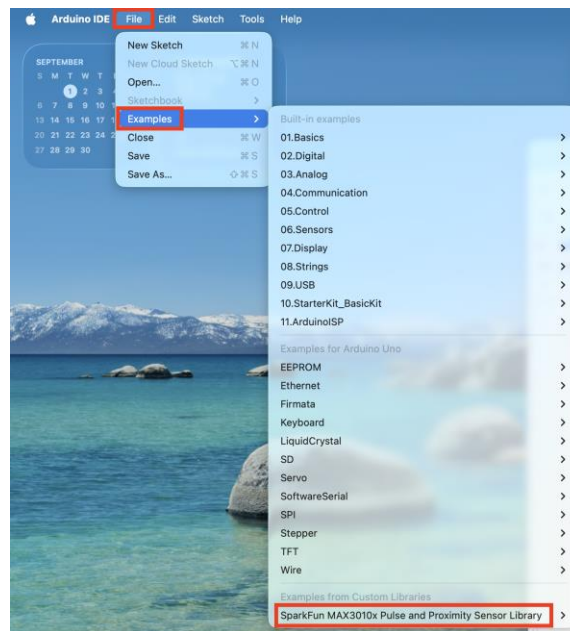
MAX30102	Arduino Uno
VIN	5V
GND	GND
SDA	A4
SCL	A5
INT	Not connected

Configuration example: (Do not follow the pins in the image below; stick to the table)



- 4) Find “**Example4_HeartBeat_Plotter**” within File → Examples → SparkFun MAX3010x Pulse and Proximity Sensor Library.

- a. Mac: “File” will be located at the menu at the very top left of your entire screen (outside of the Arduino window).



- b. This example code **plots your heartbeat by detecting changes in IR (infrared) reflectance!**

5) Press “Verify” and then “Compile”.

6) Place your finger on the small sensor area where the red LED is located.

7) Open the Serial Plotter and Serial Monitor to **see** your heartbeat (you may need to wait a few seconds with your finger steadily placed for the readings to stabilize)!

- Make sure that the Serial Monitor baud is set to 115200.
- For more consistency, tape your finger to the MAX30105 board.
- Check to see if it lines up with your pulse on your neck, it should mimic very closely (maybe with a little delay).

- d. As your heart beats, changes in blood volume in your finger cause the amount of reflected IR light to change, creating the waveform you see in the Serial Plotter.
-
- 8) Find “**Example5_HeartRate**” within File → Examples → SparkFun MAX3010x Pulse and Proximity Sensor Library.
 - a. This example code outputs instantaneous heart rate and your calculated average BPM!
 - 9) Press “Verify” and then “Compile”.
 - 10) Place your finger on the sensor and wait for a few seconds for readings that look accurate to appear on the Serial Monitor.
 - a. Make sure that the Serial Monitor baud is set to 115200.

Remember to reference the “Troubleshooting” section (page 4) if your circuit is not working/having issues.

***Again, DO NOT TAKE APART YOUR CIRCUIT.**

Congrats, you have completed all mandatory Arduino projects! Please find a member of MIND leadership to get checked off!

Optional Capstone Project: *Multi-sensor System*

Combine the potentiometer, TMP36, MAX30105, and an LED to create an **“Integrated Health Monitor System.”** Your system should measure and display heart rate and temperature in the Serial Monitor. Use the potentiometer to set an adjustable temperature threshold, which should also be displayed. The LED should blink slowly under normal conditions and blink faster if either the heart rate exceeds **100 BPM** or the measured temperature exceeds the selected threshold.

Remember to reference the “Troubleshooting” section (page 4) if your circuit is not working/having issues.

Find a member of MIND Leadership to check your work!